

this time, Tom and Mary Poppendieck published their book ‘Lean Software Development: An Agile Toolkit’. This book is still considered the all-in-one practical guide to lean software development and also gave us its seven main principles that are briefly discussed in this article.

1. Amplified learning

How can a species ensure its survival in the natural environment? The answer to this question is: natural selection. Now if only we could use some technique analogous to natural selection in software development. Sounds weird? Amplified learning is a good analogy to the biological mechanism of natural selection. In its most basic form, it can be defined as a method to initiate lateral thinking which generates alternative approaches to a single problem — the more the number of alternatives, the more is the chance of getting a better quality solution. It places huge emphasis on exploring all ideas at one’s disposal, before taking a decision.

User requirements are crucial yet complicated and evolve continuously; failing to understand these requirements can cause the project to fail. Amplified learning can help here. It also encourages open source developers to learn and implement new technologies that can help them gain experience in new ways to solve the same problem.

Amplified learning places heavy emphasis on prototyping and the concept of building an MVP, which can be directly tested as a working prototype with its key elements and features giving an insight about the market and users for the product.

Open source prototyping tools include Alva, pencil, Quant-UX, UXBOX and Akira. Almost all companies place great emphasis on fast but qualitative prototyping techniques, and some of them have developed their own methods of prototyping. For example, Apple uses an approach that it calls ‘pixel-perfect prototypes’ or ‘10-

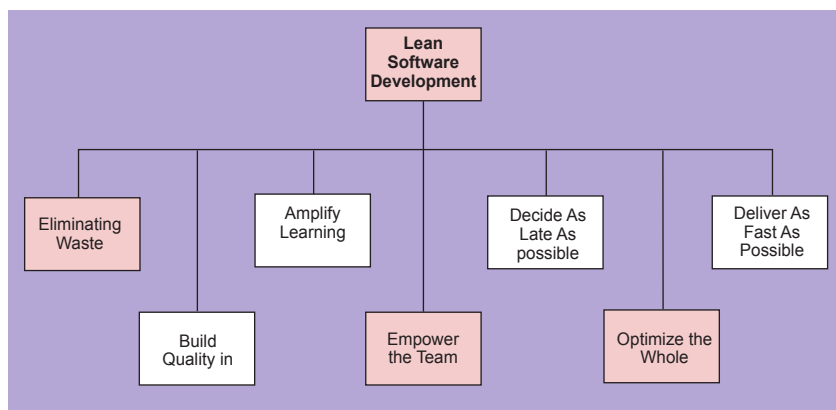


Figure 1: Lean software development

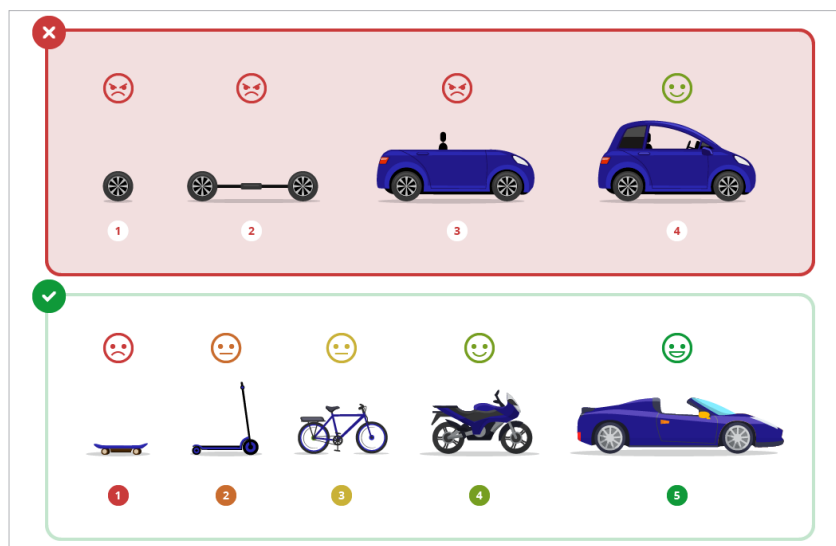


Figure 2: Minimum viable product

3-1 prototypes’. It starts with creating 10 independent concepts, from which three are selected, and from these the final product is created. Google uses a different approach called the ‘design sprint’ where a 5-day sprint (sometimes less than that) is divided into components called understand, diverge, decide, prototype and validate.

2. Delivering as fast as possible

Imagine having a revolutionary open source software idea that you are determined to make a reality, and all that hard work going to waste because someone somewhere came out with a solution to the same problem before you. That sure is a bummer. Lean MVP

software development guarantees the release of the software as fast as possible while ensuring that the first iteration is robust, minimising the after-launch bugs. Through amplified learning and deciding late we have a fool-proof plan of what we want to do and what the product should look like. So while implementing open source, developers have to only focus on evolving a working product through rapid iterations.

There is fair evidence that suggests the software that comes first to the market often becomes the leader in that domain. Also, coming with a product first helps you create a loyal customer base and get faster user feedback for better feature and requirement adoptions.